

Database Engineering for Developers & DBAs

Chapter 5 : Denormalization

Denormalization কী?

Denormalization হলো Database Design-এর এমন একটি Technique, যেখানে **Read Performance বৃদ্ধি করার জন্য** একাধিক সম্পর্কযুক্ত (Normalized) Table-কে আংশিক বা সম্পূর্ণভাবে একত্রিত (Merge) করা হয় অথবা কিছু Data ইচ্ছাকৃতভাবে Duplicate রাখা হয়।

অর্থাৎ, Normalization যেখানে Data Duplication কমানোর চেষ্টা করে, Denormalization সেখানে **দ্রুত Query Result পাওয়ার জন্য** কিছু Data Duplicate রাখতে পারে।

সহজভাবে বলতে গেলে, Denormalization হলো Performance বাড়ানোর জন্য নিয়ন্ত্রিতভাবে (Controlled) Data Duplication করা।

কেন Denormalization ব্যবহার করা হয়?

Normalization করলে Database সুসংগঠিত হয়, কিন্তু অনেক সময় একটি Report তৈরি করতে একাধিক Table JOIN করতে হয়। JOIN যত বেশি হবে, Query তত জটিল হবে এবং বড় Database-এ Response Time বেড়ে যেতে পারে।

Denormalization-এর মাধ্যমে JOIN কমিয়ে দ্রুত Data Retrieve করা যায়।

Denormalization-এর প্রধান উদ্দেশ্য

- Read Performance বৃদ্ধি করা।
- Complex JOIN কমানো।
- Report দ্রুত তৈরি করা।
- Dashboard দ্রুত Load করা।
- Analytics Query দ্রুত Execute করা।
- Large Scale Application-এ Response Time কমানো।

কখন Denormalization ব্যবহার করবেন?

Denormalization সাধারণত নিচের ধরনের System-এ ব্যবহার করা হয়—

- Data Warehouse

Database Engineering for Developers & DBAs

- Business Intelligence (BI)
- Dashboard
- Analytics System
- Reporting Database
- Data Mart
- Read-Heavy Application

কখন Denormalization ব্যবহার করবেন না?

যেসব System-এ প্রতিনিয়ত Data Update হয় এবং Data Accuracy সবচেয়ে গুরুত্বপূর্ণ—

- Banking System
- Online Banking
- Payment Gateway
- ERP
- Inventory Management
- Accounting Software
- Hospital Management System

এসব ক্ষেত্রে সাধারণত Normalization ব্যবহার করা হয়।

Denormalization কীভাবে করবেন?

Denormalization করার নির্দিষ্ট কোনো একক নিয়ম নেই। তবে সাধারণভাবে নিচের কয়েকটি পদ্ধতি অনুসরণ করা হয়।

১. Table Merge করা

একাধিক Table-এর Data একত্র করে একটি বড় Table তৈরি করা।

আগে

Customers

Customer_ID

Name

Orders

Database Engineering for Developers & DBAs

Order_ID
Customer_ID
Order_Date

পরে

Customer_Orders

Order_ID
Customer_ID
Customer_Name
Order_Date

এখানে Customer_Name Duplicate হলেও JOIN করার প্রয়োজন নেই।

২. Frequently Used Column Duplicate রাখা

যে তথ্য বারবার JOIN করে আনতে হয়, তা মূল Table-এই সংরক্ষণ করা।

উদাহরণ

Orders Table

Order_ID
Customer_ID
Customer_Name
Customer_Phone

Customer-এর নাম ও ফোন Customers Table-এ থাকলেও এখানে Duplicate রাখা হয়েছে।

৩. Summary Table তৈরি করা

Daily, Weekly বা Monthly Summary আলাদা Table-এ সংরক্ষণ করা।

উদাহরণ

Daily_Sales

Sales_Date
Total_Order
Total_Sales

এতে প্রতিবার কোটি কোটি Transaction গণনা করতে হয় না।

Database Engineering for Developers & DBAs

৪. Calculated Data সংরক্ষণ করা

যেসব Value বারবার Calculate করতে হয়, সেগুলো আগে থেকেই Store করা।

উদাহরণ

Product

Price

Quantity

Total_Amount

এখানে Total_Amount প্রতিবার Calculate না করে Store করা হয়েছে।

৫. Materialized View ব্যবহার করা

Oracle এবং PostgreSQL-এর মতো Database-এ Materialized View ব্যবহার করে Report দ্রুত Generate করা যায়।

Denormalization করার নিয়ম (Rules)

Denormalization করার সময় নিচের বিষয়গুলো অবশ্যই বিবেচনা করতে হবে—

- শুধুমাত্র Performance-এর প্রয়োজন হলে Denormalization করুন।
- অপ্রয়োজনীয় Duplicate Data তৈরি করবেন না।
- Write-এর চেয়ে Read বেশি হলে Denormalization ব্যবহার করুন।
- Update Frequency বেশি হলে সতর্ক থাকুন।
- Data Consistency বজায় রাখার ব্যবস্থা রাখুন।
- Backup ও Monitoring নিশ্চিত করুন।
- Performance Test করে সিদ্ধান্ত নিন।

Denormalization-এর সুবিধা

- Read Performance অনেক বেড়ে যায়।
- JOIN কম লাগে।
- Complex Query দ্রুত Execute হয়।
- Dashboard দ্রুত Load হয়।
- Reporting অনেক দ্রুত হয়।
- Analytics Query-এর Performance বৃদ্ধি পায়।

Database Engineering for Developers & DBAs

- Large Database-এ Response Time কমে।

Denormalization-এর অসুবিধা

- Duplicate Data তৈরি হয়।
- Storage বেশি লাগে।
- Data Update কঠিন হয়ে যায়।
- Data Inconsistency হওয়ার ঝুঁকি থাকে।
- Insert ও Update Operation ধীর হতে পারে।
- Maintenance কিছুটা জটিল হয়।

Real Production Example

ধরুন একটি E-commerce Website রয়েছে।

Normalized Database

Customers

Customer_ID	Name	City
-------------	------	------

Products

Product_ID	Product_Name	Price
------------	--------------	-------

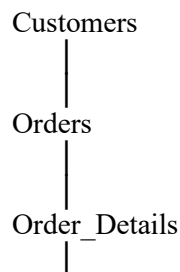
Orders

Order_ID	Customer_ID	Order_Date
----------	-------------	------------

Order_Details

Order_ID	Product_ID	Quantity
----------	------------	----------

একটি Sales Report তৈরি করতে চারটি Table JOIN করতে হবে।



Database Engineering for Developers & DBAs

|
Products

Database ছোট হলে এটি সমস্যা নয়।

কিন্তু যদি Database-এ ৫০ কোটি Order থাকে, তাহলে এই JOIN অনেক সময় নিতে পারে।

Denormalized Reporting Table

Sales_Report

Order_ID

Customer_Name

City

Product_Name

Price

Quantity

Order_Date

এখন Report তৈরির জন্য কোনো JOIN লাগছে না।

একটি Table থেকেই সব তথ্য পাওয়া যাচ্ছে।

ফলে Query অনেক দ্রুত Execute হবে।

Banking System-এ কেন Denormalization করা হয় না?

ধরুন,

Rahim-এর Account Balance **৳50,000**।

যদি একই Balance তিনটি Table-এ Duplicate করে রাখা হয় এবং Update করার সময় একটি Table Update না হয়, তাহলে তিনটি Table-এ তিন রকম Balance দেখা যেতে পারে।

এটি Banking System-এর জন্য অত্যন্ত ঝুঁকিপূর্ণ।

তাই Banking System-এ সাধারণত **Normalization** ব্যবহার করা হয়।

Database Engineering for Developers & DBAs

Data Warehouse-এ কেন Denormalization ব্যবহার করা হয়?

ধরুন,

একটি কোম্পানির CEO প্রতিদিন Sales Dashboard দেখেন।

যদি প্রতিবার ২০টি Table JOIN করতে হয়, তাহলে Report তৈরি হতে অনেক সময় লাগবে।

তাই Data Warehouse-এ সাধারণত আগে থেকেই Summary Table বা Fact Table তৈরি করে রাখা হয়।

ফলে Dashboard কয়েক সেকেন্ডের মধ্যেই Load হয়।

Normalization vs Denormalization

বিষয়	Normalization	Denormalization
মূল উদ্দেশ্য	Data Redundancy কমানো	Read Performance বাড়ানো
Data Duplication	কম	ইচ্ছাকৃতভাবে থাকতে পারে
Storage	কম লাগে	বেশি লাগে
JOIN	বেশি	কম
Read Performance	তুলনামূলক কম	বেশি
Write Performance	ভালো	কিছু ক্ষেত্রে ধীর হতে পারে
Data Consistency	বেশি	বজায় রাখতে অতিরিক্ত ব্যবস্থা দরকার
Maintenance	সহজ	তুলনামূলক জটিল
উপযুক্ত ক্ষেত্র	OLTP Database	OLAP, Reporting, Data Warehouse

সহজে মনে রাখার ট্রিক

- Normalization → Remove Duplicate Data
- Denormalization → Improve Read Performance
- Normalization → OLTP
- Denormalization → OLAP

Database Engineering for Developers & DBAs

- Normalization → Data Integrity
- Denormalization → Fast Reporting

Chapter Summary

এই অধ্যায়ে আমরা শিখেছি—

- Denormalization কী এবং কেন ব্যবহার করা হয়।
- কোন পরিস্থিতিতে Denormalization করা উচিত।
- Denormalization করার বিভিন্ন পদ্ধতি।
- এর সুবিধা ও অসুবিধা।
- বাস্তব Production Environment-এ এর ব্যবহার।
- Normalization এবং Denormalization-এর মধ্যে মূল পার্থক্য।